Foundations of Linear Algebra in Machine Learning

A Deep Dive into Vectors, Scalars, and the Geometry of Feature Space

ARMAN ASHRAF

2026

What is Linear Algebra?

Linear algebra is a branch of mathematics that studies **vectors**, **vector spaces**, **matrices**, and **linear transformations**. It provides tools to describe and manipulate **multi-dimensional data**.

Example:
A vector can represent a point in 2D space:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Here, 3 is the x-coordinate, and 4 is the y-coordinate.

A matrix can represent multiple points or a dataset:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Each row is a data point, and each column is a feature.

Why it's important in Machine Learning

Machine learning algorithms work with **data in vector and matrix form**.

- Each data point is a vector of features.
- A dataset is a matrix of all feature vectors.

Operations like **dot product**, **matrix multiplication**, and **eigenvectors** are essential for algorithms:

- **Linear regression:** Uses matrix multiplication to compute predictions.
- **PCA (Principal Component Analysis):** Uses eigenvectors to reduce dimensionality.
- **Neural networks:** Use dot products and matrix multiplications to compute activations.

Example:
Two feature vectors:

$$\mathbf{v_1} = [1, 2], \quad \mathbf{v_2} = [3, 4]$$

Dot product:

$$\mathbf{v_1} \cdot \mathbf{v_2} = 1 * 3 + 2 * 4 = 11$$

What is a Scalar

A **scalar** is a single number, not a vector or matrix.

Scalars are used as:

- Weights in linear models
- Bias terms in neural networks
- Learning rates in optimization algorithms

Example Scalars: 5, -1.2, -9

Basic Operations:

```
a = 3
b = 9

sum_scalar = a + b          # 12
product_scalar = a * b      # 27
```

What is a Vector?

A **vector** is an **ordered list of numbers** that represents **magnitude (length)** and **direction**.

Ordered means the position of numbers matters:

$$(2, 3) \neq (3, 2)$$

Example

$$\mathbf{v} = [1, 2, 4]$$

In **Machine Learning**, vectors represent **data points in feature space**, where each **axis corresponds to a feature**.

Example:

$$\mathbf{v} = [size, bedrooms, bathrooms] = [feature_1, feature_2, feature_3]$$

Example houses:

$$house_1 = [1200, 3, 2]$$

$$house_2 = [2400, 6, 4]$$

Magnitude and Direction in Machine Learning

Magnitude

The **magnitude (length)** of a vector summarizes the overall scale of the features.

Magnitude of a vector:

$$||v|| = \sqrt{x_1^2 + x_2^2 + ... + x_n^2}$$

For example:

$$house_1 = [1200, 3, 2]$$

$$house_2 = [2400, 6, 4]$$

Since every value in **house$_2$ is larger**, its **magnitude is larger**, meaning the house is overall **bigger**.

Direction

The **direction** of a vector represents the **relationship between features**.

It captures the **proportions of the features**, not the size.

Example:

$$house_1 = [1200, 3, 2]$$

$$house_2 = [2400, 6, 4]$$

Both houses follow the same proportion:

$$1200 : 3 : 2 = 2400 : 6 : 4$$

So they point in the **same direction in feature space**.

Even though **house$_2$ is bigger**, the **pattern of features is identical**.

Why Direction Matters in Machine Learning

Many ML algorithms care about **direction more than magnitude**.

Example: **Cosine Similarity**

Cosine similarity measures the **angle between two vectors**.

If the angle is small → vectors are similar.

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{||a||\,||b||}$$

This is widely used in:

- Text embeddings
- Image embeddings
- Recommendation systems
- Semantic search

If two vectors have the **same direction**, cosine similarity **1**.

Scalar and Vector Projection

A **projection** is like the **shadow of one vector on another vector**.

Imagine shining a light above vector **a** so its shadow falls onto vector **b**.

Scalar Projection

The **scalar projection** is a **single number**.

It tells us **how much of vector a lies in the direction of vector b**.

Formula

$$scalar_projection = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||}$$

It represents the **length of the shadow**.

Intuition Example

Imagine pulling a box with a rope at an angle.

Your total force is not fully moving the box forward.

The **scalar projection** tells you **how much of your force actually moves the box forward**.

Vector Projection

The **vector projection** is the **shadow itself**, represented as a vector.

It has:

- the **length of the scalar projection**
- the **direction of vector b**

Formula

$$vector_projection = \left(\frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||^2}\right)\mathbf{b}$$

Why Projection Matters in Machine Learning

Projection is used to **simplify high-dimensional data**.

Dimensionality Reduction (PCA)

Principal Component Analysis **projects data onto important directions**.

Example intuition:

A **3D object casting a shadow on a wall**.

3D object → shadow on wall → 2D representation

The shadow **keeps important structure but removes extra dimensions**.

PCA works the same way.

It projects high-dimensional data onto **principal directions that contain the most information**.

Orthogonality

If the projection is **zero**, the vectors are **orthogonal (90° angle)**.

$$\mathbf{a} \cdot \mathbf{b} = 0$$

This means the vectors are **completely independent**.

In ML, this means:

- Features contain **no shared information**.

Unit Vector

A **unit vector** is a vector with **magnitude = 1**.

It represents **direction only**, without size.

Formula

To convert any vector into a unit vector:

$$\hat{v} = \frac{v}{||v||}$$

Example:

If

$$v = [3, 4]$$

Magnitude:

$$||v|| = 5$$

Unit vector:

$$\hat{v} = \left[\frac{3}{5}, \frac{4}{5}\right]$$

Now the vector keeps the **same direction but its length becomes 1**.

Why Unit Vectors Matter in Machine Learning

Unit vectors are used when we only care about **direction**.

Examples:

- **Cosine similarity**
- **Embeddings comparison**
- **Normalization of feature vectors**

In embeddings (like text or images), vectors are often **normalized to unit vectors** so that comparison depends only on **direction (semantic meaning)**, not magnitude.

Dot Product

The **dot product** multiplies the corresponding elements of two vectors and then adds the results.

For vectors:

$$\mathbf{a} = [a_1, a_2, a_3], \quad \mathbf{b} = [b_1, b_2, b_3]$$

Dot product:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Example

$$\mathbf{a} = [1, 2, 3]$$

$$\mathbf{b} = [4, 5, 6]$$

$$\mathbf{a} \cdot \mathbf{b} = (1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Geometric Meaning

The dot product also measures **how aligned two vectors are**:

$$\mathbf{a} \cdot \mathbf{b} = ||a||\,||b|| \cos(\theta)$$

Where:

- $||a||$ and $||b||$ are magnitudes
- θ is the angle between the vectors

If:

- $\theta = 0^\circ \rightarrow$ vectors point in the **same direction**
- $\theta = 90^\circ \rightarrow$ dot product = **0** (orthogonal)
- $\theta = 180^\circ \rightarrow$ vectors point in **opposite directions**

Machine Learning Use

Dot products are used in:

- **Cosine similarity**
- **Neural networks**
- **Linear regression**
- **Embedding similarity**

Example cosine similarity formula:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{||a||\,||b||}$$

If vectors point in the **same direction**, cosine similarity **1**, meaning the vectors represent **similar information**.

Cross Product

The **cross product** is an operation between **two vectors in 3D space** that produces a new vector that is **perpendicular (orthogonal) to both vectors**.

For vectors:

$$\mathbf{a} = [a_1, a_2, a_3]$$

$$\mathbf{b} = [b_1, b_2, b_3]$$

The cross product is:

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

Example

Let:

$$\mathbf{a} = [1, 0, 0]$$

$$\mathbf{b} = [0, 1, 0]$$

Then:

$$\mathbf{a} \times \mathbf{b} = [0, 0, 1]$$

The resulting vector is **perpendicular to both vectors**.

Cross products are mainly used in:

- **3D graphics**

- **physics**
- **robotics**

What is a Matrix

A **matrix** is a table of numbers arranged in **rows and columns**.

Example matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Rows = horizontal
Columns = vertical

Matrices are used to represent:

- datasets
- linear transformations
- neural network weights

Identity Matrix

An **identity matrix** is a square matrix where:

- all diagonal elements are **1**
- all other elements are **0**

Example:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Property:

$$AI = IA = A$$

Multiplying by the identity matrix **does not change the matrix**.

Diagonal Matrix

A **diagonal matrix** is a square matrix where **all elements outside the main diagonal are zero**.

Example:

$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

Diagonal matrices are important in:

- **Eigenvalue decomposition**
- **SVD**
- **PCA**

Symmetric Matrix

A **symmetric matrix** is a matrix that is equal to its transpose.

$$A = A^T$$

Example:

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 3 & 5 & 6 \\ 4 & 6 & 7 \end{bmatrix}$$

Properties:

- $A_{ij} = A_{ji}$
- Appears in **covariance matrices**

Symmetric matrices are very important in **machine learning**.

Rank of a Matrix

The **rank** of a matrix represents the **number of linearly independent rows or columns**.

In simple terms:

Rank tells us **how much unique information the matrix contains**.

Example

If a matrix is:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Row 2 is:

$$2 \times Row1$$

So the rows are **dependent**.

Therefore:

$$rank(A) = 1$$

Intuition

Redundant rows do not increase rank.

Example:

```
Row1 = [1,2]
Row2 = [2,4]
```

Row2 adds **no new information**.

Dimensional Interpretation

If a **3×3 matrix has rank $= 2$**, it means the data lies in a **2-dimensional plane instead of full 3D space**.

This idea is important for:

- **Dimensionality reduction**
- **PCA**
- **Feature redundancy detection**

Trace of a Matrix

The **trace** of a matrix is the **sum of elements on the main diagonal**.

Rule: The matrix must be **square**.

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Trace:

$$tr(A) = 1 + 5 + 9 = 15$$

Determinant (Geometric Intuition)

The **determinant** measures how a matrix **scales space**.

Imagine a **rubber square** (2D) or **rubber cube** (3D).

When you multiply by a matrix, the matrix **stretches, compresses, or flips the shape**.

In 2D

The determinant tells us the **area scaling factor**.

Example:

If the original square area = **1**
and determinant = **5**

Then the transformed shape (a parallelogram) has:

Area = **5**

In 3D

The determinant tells us the **volume scaling factor**.

Example:

If the original cube volume = **1**
and determinant = **10**

Then the transformed shape (parallelepiped) has:

Volume = **10**

Important Property

If:

$$det(A) = 0$$

The transformation **collapses the space**.

Example:

3D → 2D plane
2D → line

This means the matrix **loses information**.

This concept is important for understanding **rank, invertibility, and dimensionality reduction**.

Linear Transformation

A **linear transformation** is a function that maps **vectors to vectors** while preserving two important properties.

1. Additivity

$$T(u + v) = T(u) + T(v)$$

2. Scalar Multiplication

$$T(cu) = cT(u)$$

Where:

- T = linear transformation
- u, v = vectors
- c = scalar

Example

Suppose a transformation scales vectors by 2.

If:

$$v = [1, 2]$$

Then:

$$T(v) = [2, 4]$$

The transformation **preserves vector structure**.

Use Cases in AI

Images can be represented as **matrices of pixel values**.

Linear transformations can:

- Rotate images
- Scale images
- Apply shearing
- Perform geometric transformations

Neural networks are essentially **chains of linear transformations plus non-linear functions**.

Kernel (Null Space)

The **kernel** (or **null space**) of a matrix is the set of all vectors that become the **zero vector after transformation**.

Mathematically:

$$Ax = 0$$

This means:

When matrix A transforms vector x, the output becomes the **zero vector**.

Example with Feature Redundancy

Suppose we have two features:

- x_1 = height in meters
- x_2 = height in centimeters

Since:

$$1 \text{ meter} = 100 \text{ cm}$$

We know:

$$x_2 = 100x_1$$

So these two features contain the **same information**.

Represent as a Vector

A data point might look like:

$$x = \begin{bmatrix} h \\ 100h \end{bmatrix}$$

Now define matrix:

$$A = \begin{bmatrix} 100 & -1 \end{bmatrix}$$

Apply Transformation

$$Ax = \begin{bmatrix} 100 & -1 \end{bmatrix} \begin{bmatrix} h \\ 100h \end{bmatrix}$$

Compute:

$$100h - 100h = 0$$

So:

$$Ax = 0$$

What Happened?

The vector

$$\begin{bmatrix} h \\ 100h \end{bmatrix}$$

becomes **zero after transformation**.

Therefore it belongs to the **kernel of** A.

Machine Learning Intuition

In ML this means:

The model detects that

```
height_meters
height_cm
```

represent the **same information**.

So the model **removes that redundant direction**.

That direction becomes part of the **kernel**.

Meaning:

Moving along that direction **does not change the output**.

Range (Column Space / Image)

The **range** of a transformation is the set of all **possible output vectors** that the transformation can produce.

Mathematically:

$$Range(A) = \{Ax \mid x \in \mathbb{R}^n\}$$

It is also called the **column space of the matrix**.

AI Intuition

Range tells us **what kind of outputs a model can generate**.

Example:

Imagine a **Generative AI model** trained only on **cat images**.

Its **range** is limited to:

- cat-like images
- cat variations

It **cannot generate dogs** because dogs are **outside the range of the model**.

Eigenvectors and Eigenvalues

Usually when a matrix transforms a vector:

- the **direction changes**
- the **length changes**

But some special vectors behave differently.

Eigenvectors

Eigenvectors are vectors whose **direction does not change** after transformation.

They only **stretch or shrink**.

Eigenvalues

The amount of **stretching or shrinking** is called the **eigenvalue**.

Mathematically:

$$Av = \lambda v$$

Where:

- A = matrix (transformation)
- v = eigenvector
- λ = eigenvalue

Example

If:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Then vector

$$v = [1, 0]$$

becomes

$$Av = [2, 0]$$

The direction is the same.

Only the **length changed by factor 2**.

So:

- v is an **eigenvector**
- $\lambda = 2$ is the **eigenvalue**

Importance in Machine Learning

Eigenvectors are used in:

- **PCA (Principal Component Analysis)**
- **Dimensionality reduction**
- **Covariance matrix decomposition**

They help find the **most important directions in data**.

Gram–Schmidt Process

The **Gram–Schmidt process** converts a set of **linearly independent vectors** into **orthogonal (or orthonormal) vectors**.

In simple words:

It takes vectors pointing in different directions and converts them into **perpendicular vectors that span the same space**.

Why We Need It

Working with perpendicular vectors makes many computations easier.

With orthogonal vectors:

- projections become easier
- calculations simplify
- matrix decompositions become possible

Gram–Schmidt is used in:

- **QR decomposition**
- **numerical linear algebra**
- **machine learning algorithms**

Intuition of Gram–Schmidt

Suppose we start with two vectors:

$$v_1, v_2$$

These vectors may not be perpendicular.

The Gram–Schmidt process converts them into:

$$u_1, u_2$$

such that:

- u_1 and u_2 are **perpendicular**
- they **span the same space** as v_1 and v_2

So we keep the **same information**, but in a **cleaner orthogonal coordinate system**.

Inverse of a Matrix

The **inverse** of a square matrix A is another matrix A^{-1} such that:

$$AA^{-1} = A^{-1}A = I$$

where I is the **identity matrix**.

Intuitively, the **inverse matrix undoes the transformation performed by** A.

Example Matrix

Consider the matrix:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

If a vector is:

$$x = \begin{bmatrix} a \\ b \end{bmatrix}$$

then the transformation becomes:

$$Ax = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a + 2b \\ 3a + 4b \end{bmatrix}$$

So the matrix **mixes the features together**.

ML Intuition of the Inverse

Suppose:

$$y = Ax$$

The inverse allows us to recover the original vector:

$$x = A^{-1}y$$

So the inverse **separates the mixed information**.

Example in Machine Learning

Imagine two original features:

- x_1 = temperature
- x_2 = humidity

But a sensor outputs **mixed signals**:

$$y_1 = x_1 + 2x_2$$

$$y_2 = 3x_1 + 4x_2$$

This can be written as:

$$y = Ax$$

where

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

To recover the original features:

$$x = A^{-1}y$$

The inverse **undoes the mixing of features**.

Real AI Example: Signal Separation

This idea appears in **Blind Source Separation**.

Example:

Two microphones record two people speaking.

Each microphone hears a mixture:

```
mic1 = voice1 + voice2
mic2 = 2*voice1 + 3*voice2
```

This is a matrix transformation.

Using the **inverse matrix**, we can separate the voices.

Applications include:

- Speech processing
- Audio separation
- Signal processing

Orthogonal Matrix

An **orthogonal matrix** is a square matrix whose columns (and rows) are **orthogonal unit vectors**.

Orthogonal means:

- vectors are **perpendicular**
- their **dot product is zero**

Unit vector means:

- the vector's **magnitude is 1**

Important Property

For an orthogonal matrix Q:

$$Q^T Q = I$$

This means:

$$Q^{-1} = Q^T$$

So the **inverse of an orthogonal matrix is its transpose**.

This property makes computations **very efficient**.

Characteristic Equation

The **characteristic equation** is used to find the **eigenvalues** of a matrix.

For a square matrix A:

$$\det(A - \lambda I) = 0$$

Where:

- λ = eigenvalue
- I = identity matrix
- det = determinant

Intuition

The characteristic equation tells us **how much the matrix stretches or shrinks vectors along special directions (eigenvectors).**

Covariance Matrix

A **covariance matrix** shows how variables in a dataset **change together**.

For a dataset with multiple features:

- **Diagonal elements** represent the **variance of each feature**.
- **Off-diagonal elements** represent the **covariance between two features**.

Example:

$$C = \begin{bmatrix} Var(x_1) & Cov(x_1, x_2) \\ Cov(x_2, x_1) & Var(x_2) \end{bmatrix}$$

ML Intuition

The covariance matrix helps us understand:

- which features **vary together**
- which features are **independent**

It is heavily used in:

- **Principal Component Analysis (PCA)**
- **Dimensionality reduction**
- **Feature analysis**

Diagonalization

Diagonalization means converting a complicated matrix into a **diagonal matrix**, where most values are zero except the diagonal.

This makes many computations **much easier**.

Mathematical Form

A matrix A can be diagonalized as:

$$A = PDP^{-1}$$

Where:

- A = original matrix
- D = diagonal matrix (contains eigenvalues)
- P = matrix of eigenvectors
- P^{-1} = inverse of P

ML Intuition

Imagine a dataset with **100 features**.

Many features are correlated and messy.

Diagonalization helps separate the data into **independent directions**.

This allows algorithms to:

- analyze patterns more easily
- reduce dimensionality
- speed up computations

One-Line Intuition

Diagonalization = converting a complicated matrix into a simple diagonal form so computations become easier.

Spectral Decomposition

Simple Idea

Spectral decomposition breaks a matrix into parts using its **eigenvalues** and **eigenvectors**.

Instead of working with a complicated matrix, we represent it using its **eigendirections**.

So the idea is:

Matrix = combination of its eigenvectors and eigenvalues.

Mathematical Form

For a **symmetric matrix** A:

$$A = Q\Lambda Q^T$$

Where:

- Q = matrix of eigenvectors
- Λ = diagonal matrix of eigenvalues
- Q^T = transpose of Q

Intuition

The matrix transformation can be understood as:

1. Rotate into eigenvector directions
2. Scale by eigenvalues
3. Rotate back

LU Decomposition

Definition

LU decomposition breaks a matrix into two matrices:

$$A = LU$$

Where:

- L = **lower triangular matrix**
- U = **upper triangular matrix**

Why It Is Useful

LU decomposition is used to **solve linear systems efficiently**.

Example:

$$Ax = b$$

Instead of solving directly, we solve:

1. $Ly = b$
2. $Ux = y$

This makes computations faster for large systems.

QR Decomposition

Definition

QR decomposition breaks a matrix into:

$$A = QR$$

Where:

- Q = orthogonal matrix
- R = upper triangular matrix

Why It Is Useful

QR decomposition is commonly used for:

- solving **least squares problems**
- **linear regression**
- numerical stability in computations

Singular Value Decomposition (SVD)

Definition

Singular Value Decomposition breaks a matrix into three matrices:

$$A = U\Sigma V^T$$

Where:

- U = left singular vectors
- Σ = diagonal matrix of singular values
- V^T = right singular vectors

Simple Intuition

Any matrix transformation can be seen as:

```
Rotate → Stretch/Shrink → Rotate
```

- V^T rotates the input space
- Σ stretches or shrinks it
- U rotates it again

Why We Need SVD

1. Dimensionality Reduction

If a dataset has **many features**, most information may lie in only a few directions.

SVD keeps the **largest singular values** and removes the rest.

This reduces data size while keeping important information.

Used in:

- **PCA**
- **image compression**
- **noise reduction**
- **recommendation systems**